kata – Describe katas with pictograms

Lukas Heindl
oss.heindl+latex@protonmail.com
Github: https://github.com/atticus-sullivan/kata/

0.1.0 from 2026/03/23

The `kata` package provides a compact and structured way to typeset martial arts katas[1] (forms) in LaTeX. It models a kata as a sequence of steps, each combining the stance hand/foot technique, and transition direction, and supports customizing the resulting pictograms for clear visual representation.

Requires: KOMA-documentclass

1 Introduction

In many martial arts such as Karate, a kata is a predefined sequence of movements [2].

The `kata` package aims to provide a simple, structured way to describe katas in LaTeX. Each step of a kata is expressed as a step consisting of

1. the stance with direction
2. a technique with direction
3. the direction of movement or rotation into the next step

The package also offers customizing the resulting pictograms, allowing visual annotation of sequences, such as highlighting repetitions or emphasizing specific actions.

2 Usage

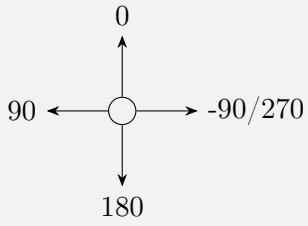
Load this package as `\usepackage{kata}`.

Note this package uses functionality provided by the KOMA-classes (mostly `\marginline`), so it only works with such a documentclass.

[1] https://en.wikipedia.org/wiki/Kata
[2] https://en.wikipedia.org/wiki/Kata

Note: Concept of *direction*

One quite important concept is the concept of the *direction*. Some commands such as `\waza`[→P. 3], `\dir`[→P. 3], and `\stance`[→P. 3] refer to a direction in the range of -360 to 360 degrees. Also some pgfkeys take a direction as an argument.
This package consistently defines the direction as follows:




```
\begin{kata}[⟨pgfkeys⟩]{⟨title⟩}{⟨margin⟩}[⟨belt-color⟩]
  ⟨environment content⟩
\end{kata}
```

The container which holds an entire kata. Within normal LaTeXcode may be typeset but mostly the content usually consists of `step` environments.
This will also create a section named ⟨*title*⟩. It also adds a marginline which is thought to indicate for which belt this kata is required. So the marginline consists of

- an arrow pointing down
- ⟨*margin*⟩
- a drawing showing a belt. The color by default will be ⟨*margin*⟩ but it may be overwritten with ⟨*belt-color*⟩

  If ⟨*margin*⟩ is empty and ⟨*belt-color*⟩ is not given, no marginline will be drawn

The `[⟨pgfkeys⟩]` may be used to set keys in the `/kata/cmd/kata` space.
After all steps have been drawn it will add a number of phantom-steps trying to generate an even looking grid of pictograms. For this `/kata/cmd/kata/width`[→P. 3] should be set to the number of pictograms fitting one line.

```
\begin{step}[⟨pgfkeys⟩]
  ⟨environment content⟩
\end{step}
```

Draws a pictogram representing one step in the kata. ⟨*pgfkeys*⟩ may set any key from the space `/kata/cmd/step`
The body will be the content of a `tikzpicture` so normal LaTeXcode won't work as expected.
Usually you'll use `\waza`[→P. 3], `\stance`[→P. 3] and `\dir`[→P. 3] to populate the step with contents. Note that all of these commands may be present (multiple times) but they don't need to be present all.

If you're drawing manually with tikz, note the node `head` exists and is at the center. Also be careful to not draw out of the normal bounds, ototherwise the alignment will be off.

`\waza[⟨side⟩]{⟨content⟩}{⟨dir⟩}[⟨pgfkeys⟩]`

Use this command to draw a technique with its direction.

⟨***side***⟩ may specify the side (R/L or right/left) which executes the technique

⟨***content***⟩ the text describing the action

⟨***dir***⟩ the direction in which to execute the action

⟨***pgfkeys***⟩ set any of the keys in the `/kata/cmd/content` space. Notably is the `/kata/cmd/content/flip` key.

`\stance[⟨side⟩]{⟨content⟩}{⟨dir⟩}[⟨pgfkeys⟩]`

Use this command to draw the technique+direction for the stance.

⟨***side***⟩ may specify the side (R/L or right/left) which executes the technique

⟨***content***⟩ the text describing the action

⟨***dir***⟩ the direction in which to execute the action

⟨***pgfkeys***⟩ set any of the keys in the `/kata/cmd/content` space. Notably is the `/kata/cmd/content/flip` key.

`\dir[⟨pgfkeys⟩]{⟨dirA⟩}{⟨dirB⟩}`

Use this command to draw an indicator indicating the direction/turn to transition to the next step.

⟨***dirA***⟩ is where the movement starts (in degrees -360 to 360)

⟨***dirB***⟩ is where the movement ends (in degrees -360 to 360)

⟨***pgfkeys***⟩ set any of the keys in the `/kata/cmd/content` space. Notably is the `/kata/cmd/content/flip` key which causes the arc to be drawn in the other direction.

Note for ⟨*dirA*⟩ the string *center* is also eligible. This causes the arrow to start at the center of the circle, indicating only a direction not a turn.

`/kata/cmd/content/flip` (bool)

Flip the "thing". Usually this leads to the text-node being placed in the other direction in order to avoid collisions.

`/kata/cmd/step/first` (bool)

Specify this is the first step of the kata. This enables various things which mostly affect the spacing.

`/kata/cmd/step/phantom` (style)

Activate this style to hide this step. The step still occupies space but is not visible.

`/kata/cmd/kata/width` (initially: 6)

The number of steps fitting on one line.

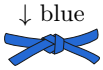
2.1 Simple Example

Note this code is "run" after the box showing the code. Because of the use of `\marginline` it was not possible to wrap this in `dispExample*`. Also this way the `\section` command will leak and insert a new section to this documentation. <*doc>

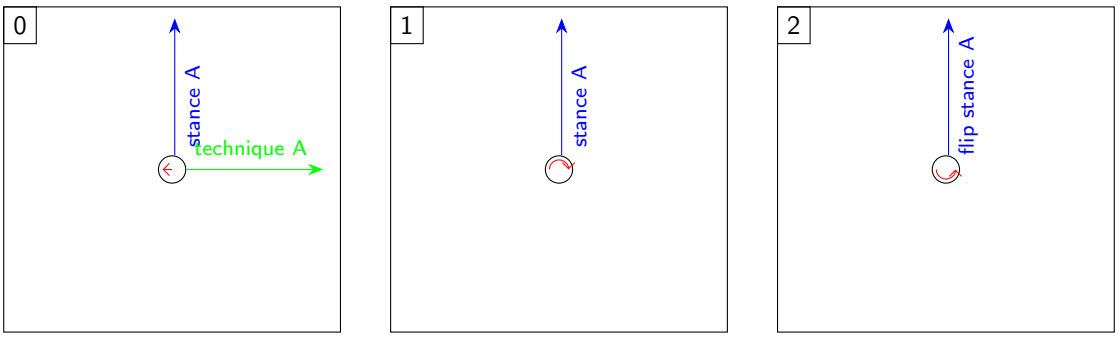
```
\begin{kata}[width=3]{Simple Example}{blue}
  Some explanation -- normal \LaTeX code
  \parindent=0pt

  \begin{step}
    \stance{stance A}{0}
    \waza{technique A}{-90}
    \dir{center}{90}
  \end{step}
  \begin{step}
    \stance{stance A}{0}
    \dir{90}{-90}
  \end{step}
  \begin{step}
    \stance[flip]{stance A}{0}
    \dir[flip]{90}{-90}
  \end{step}
\end{kata}
```

3 Simple Example

↓ blue



Some explanation – normal LaTeXcode



</doc>

3.1 Customizations

There are several ways the pictograms can be customized. They are all drawn with tikz so you can simply change the definition of some styles for the adjustments. In order to make common adjustments easier there are some helpers for this. Think of it in multiple levels. With each level you can customize more, but also you need to know more about pgfkeys.

`\kataStepCnt{⟨counter⟩}`

Formats the counter holding the number of the current step. Adjust this with e.g. `\RenewDocumentCommand` or `renewcommand` to change how the counter is displayed.

3.1.1 Level 1 – Beginner

In this level you can only customize in ways which have been prepared in advance already. For this you can use the `\katastyle` command similar like you would use `\tikzset`. Note that the settings are bound to the scope. So if you change the setting within a `kata` → P. 2 environment, the change will only persist until the environment is closed.

`\katastyle{⟨pgfkeys-def⟩}`

Allows to set keys in the `/kata/styles` space. The keys you can set are described in this section

`/kata/styles/border/right=⟨color⟩` (style)

sets the border such that it has indicators (arrows) pointing to the right. ⟨*color*⟩ specifies a color for the border. Also this will shade the background with 5% of the specified color.

`/kata/styles/border/left=⟨color⟩` (style)

similar like `/kata/styles/border/left` but with indicators pointing to the left.

`/kata/styles/border/up=⟨color⟩` (style)

similar like `/kata/styles/border/left` but with indicators pointing upwards.

`/kata/styles/border/down=⟨color⟩` (style)

similar like `/kata/styles/border/left` but with indicators pointing downwards.

`/kata/styles/border/reset=⟨color⟩` (style)

resets the changes done by the keys in the `/kata/styles/border` space.

`/kata/styles/step/kiai=⟨direction⟩` (style)

add an indicator for *Kiai* in the specified direction.

The following (x)-color definitions are used when drawing the pictograms:

`KataColorWaza` used for the arrow/path showing a technique with its direction

`KataColorFeet` used for the arrow/path showing the stance with its direction

`KataColorDir` used for the indicator showing the direction of the next move/turn

You can set these for example via `\colorlet`. Just make sure the modification takes place after this package got loaded.

3.1.2 Level 2 – Advanced

`/kata/customize/stepCnt` (style)

Applies to the indicator with the step counter.
Style which is applied after what is required for the default placement. You can redefine this style to adjust the appearance (in theory also the placement – everything what tikz allows).

`/kata/customize/border` (style)

Applies to the border around the pictogram. Note: This rectangle is drawn on the background so there is no issue in using `fill`.
Style which is applied after what is required for the default placement. You can redefine this style to adjust the appearance (in theory also the placement – everything what tikz allows).

`/kata/customize/head` (style)

Applies to the node showing the center which symbolizes the head.
Style which is applied after what is required for the default placement. You can redefine this style to adjust the appearance (in theory also the placement – everything what tikz allows).

`/kata/customize/dir` (style)

Applies to the indicator for the direction of the next move/turn.
Style which is applied after what is required for the default placement. You can redefine this style to adjust the appearance (in theory also the placement – everything what tikz allows).

Helpers: Border For the `/kata/customize/border` there are some helper styles which can be used (nor (re)defined) to aid in the customization.

`/kata/border/arrowRL=`⟨*direction*⟩ (style)

Draws two arrows on the path. If the path is a rectangle, one will be on the top, one on the bottom. Both point in the same direction specified with ⟨*direction*⟩.
This way arrows pointing right/left are drawn correctly (hance the name).

`/kata/border/arrowUD=`⟨*direction*⟩ (style)

Draws two arrows on the path. If the path is a rectangle, one will be on the right, one on the left. Both point in the same direction specified with ⟨*direction*⟩.
This way arrows pointing up/down are drawn correctly (hance the name).

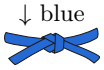
3.2 Extended Example

Note this code is "run" after the box showing the code. Because of the use of `\marginline` it was not possible to wrap this in `dispExample*`. Also this way the `\section` command will leak and insert a new section to this documentation. <*doc>

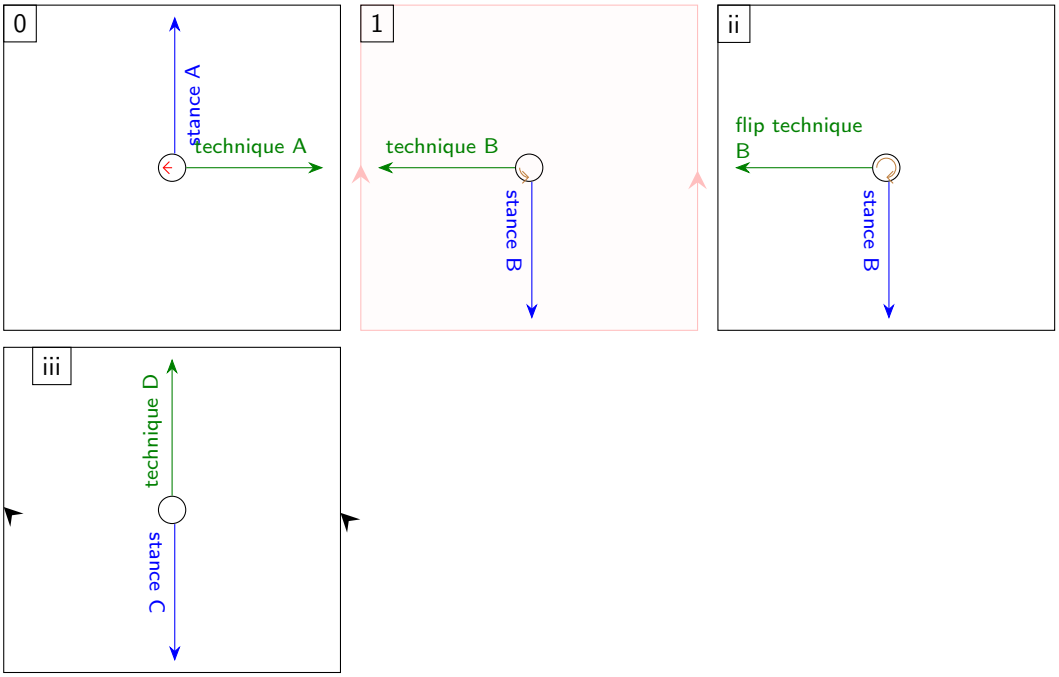
```
\begin{kata}[width=3]{Extended Example}{blue}
  Some explanation -- normal \LaTeX code
  \parindent=0pt

  \colorlet{KataColorWaza}{green!50!black}
  \begin{step}
    \stance{stance A}{0}
    \waza{technique A}{-90}
    \dir{center}{90}
  \end{step}
  \colorlet{KataColorDir}{brown}
  \katastyle{border/up=pink}
  \begin{step}
    \stance{stance B}{180}
    \waza{technique B}{90}
    \dir{90}{180}
  \end{step}
  \katastyle{border/reset}
  \renewcommand\kataStepCnt[1]{\roman{#1}}
  \begin{step}
    \stance{stance B}{180}
    \waza[flip]{technique B}{90}
    \dir[flip]{90}{180}
  \end{step}
  \pgfkeys{
    /kata/customize/stepCnt/.style={xshift=1em},
    /kata/customize/border/.style={/kata/border/arrowUD=45},
  }
  \begin{step}
    \stance{stance C}{180}
    \waza{technique D}{0}
  \end{step}
\end{kata}
```

4 Example

↓ blue



Some explanation – normal LaTeXcode



</doc>

5 Implementation

```
\RequirePackage{memoize}
\RequirePackage{pgfkeys}
\RequirePackage{coloredbelts}
\RequirePackage{tikz}
\usetikzlibrary{calc,positioning,fit,arrows.meta}
\usetikzlibrary{decorations.markings,decorations.pathmorphing,backgrounds}

\RequirePackage{scrbase}
\AtBeginDocument{%
  \@ifundefined{KOMAClassName}{%
    \PackageError{mypackage}
      {This package requires a KOMA-Script class}
      {Use scrartcl, scrreprt, or scrbook.}%
  }{}
}
```

5.1 Internals

A counter for each pictogram to enable referencing.

```
\newcounter{kata@step}[section]
```

5.1.1 Keys for Argparsing

define pgfkeys for arguments to defined commands (argparsing)

```
\newif\ifkata@flip
\newif\ifkata@first
\pgfkeys{
```

Arguments for content commands like `stance`

```
  /kata/cmd/content/.cd,
  flip/.is if=kata@flip,
  flip/.default=true,
```

Arguments for step env

```
  /kata/cmd/step/.cd,
  /kata/cmd/step/.search also={/kata/styles/step},
  first/.is if=kata@first,
  first/.default=true,
  phantom/.style={
    /kata/customize/border/.style={draw=none},
    /kata/head/.style={draw=none},
    /kata/customize/stepCnt/.style={draw=none,transparent},
  },
  phantom/.meaning to context,
```

Arguments for kata env

```
  /kata/cmd/kata/.cd,
  width/.initial=6,
  width/.value required,
}
```

5.1.2 Internal pgf/TikZ Styles

set pgf keys used when drawing the pictograms `/kata` namespace

style for the node placed at the center representing the head

```
\pgfkeys{
  /kata/.cd,
  head/.style={circle,draw=#1,inner xsep=0.333em},
  head/.meaning to context,
```

style for the arrow/path showing the stance takes two arguments to nudge the position so the path is not drawn directly on top of other paths (the *waza*)

```
  stance/.style 2 args={
    KataColorFeet,
    -{Stealth[scale=1.2]},
    transform canvas={xshift=#1,yshift=#2}
  },
  stance/.meaning to context,
```

style for the arrow/path showing a technique

```
  waza/.style={KataColorWaza,-{Stealth[scale=1.2]}},
  waza/.meaning to context,
```

style for the arrow/path indicating the next movement/direction

```
  dir/.style={KataColorDir,-{Straight Barb}},
  dir/.meaning to context,
```

style(s) for the node containing the description for the technique of the stance split up in `NodeCommon`, `stanceNo` and `stanceNodeFlipped` so when the user requests to flip the node (e.g. to avoid collision with other text) the placement can be adjusted

```
stanceNodeCommon/.style={text width=1.8cm,midway,sloped,font=\sffamily\scriptsize},
stanceNodeCommon/.meaning to context,
stanceNode/.style={/kata/stanceNodeCommon,below,},
stanceNode/.meaning to context,
stanceNodeFlipped/.style={/kata/stanceNodeCommon,above,},
stanceNodeFlipped/.meaning to context,
```

style(s) for the node containing the description for a technique split up in `NodeCommon`, `stanceNo` and `stanceNodeFlipped` so when the user requests to flip the node (e.g. to avoid collision with other text) the placement can be adjusted

```
wazaNodeCommon/.style={text width=1.8cm,midway,sloped,font=\sffamily\scriptsize},
wazaNodeCommon/.meaning to context,
wazaNode/.style={/kata/wazaNodeCommon,above,},
wazaNode/.meaning to context,
wazaNodeFlipped/.style={/kata/wazaNodeCommon,below,},
wazaNodeFlipped/.meaning to context,
```

style for the border enclosing the pictogram takes the color as argument

```
border/.style={
  rectangle,
  draw=#1,
  fit=(output),
  inner sep=0pt,outer sep=0pt,inner xsep=0.2em
},
border/.meaning to context,
```

style for the node showing the number of the current step

```
stepCnt/.style={
  outer sep=0pt,
  at={(t1.north west)},
  below right,
  rectangle,
  draw,
  font=\sffamily\small
},
stepCnt/.meaning to context,
}
```

5.1.3 Internal Elements of the Pictograms

`\kata@nothing` ensure the box is always at least the same size (internal command + always placed)

```
\NewDocumentCommand{\kata@nothing}{}{
  \draw[draw=none] (head) -- ++(0:2.15) -- ++(180:4.3) -- ++(90:2.15) -- ++(-90:4.3);
}
```

`\kata@head` draw the circle symbolizing the head (internal command + always placed)

```
\NewDocumentCommand{\kata@head}{}{
  \node[/kata/head,/kata/customize/head] (head) {};
}
```

5.1.4 Helpers

`kata@requirestep` enforces something is run within the step environment (helper)

```
\newif\ifkata@instep
\NewDocumentCommand{\kata@requirestep}{m}{%
  \ifkata@instep\else
    \PackageError{kata}{#1 used outside step environment}{}
  \fi
}
```

`kata@requirekata` enforces something is run within the kata environment (helper)

```
\newif\ifkata@inkata
\NewDocumentCommand{\kata@requirekata}{m}{%
  \ifkata@inkata\else
    \PackageError{kata}{#1 used outside kata environment}{}
  \fi
}
```

5.1.5 Draw the Marginline

`\kata@beltMargin`

```
\NewDocumentCommand{\kata@beltMargin}{mm}{%
  \marginline{%
    \centering\footnotesize%
    \Ifstr{#1}{}{}{%
      $\downarrow$ #1%
    }%
```

only linebreak if both are non-empty

```
    \Ifstr{#1}{}{}{\Ifstr{#2}{}{}{\\}}%
    \Ifstr{#2}{}{}{%
      \ColorBelt*[scale=0.1]{#2}%
    }%
  }%
}
```

5.1.6 Environments

`kata` (*env.*) this environment is basically a container for a sequence of steps. Arguments:

1. optional: keys from `/kata/cmd/kata`
2. kata name
3. for which belt / marginline

4. optional: belt color (for drawing), if empty -> not drawn, if not present use arg 3

```
\NewDocumentEnvironment{kata}{O{}mmo}{%
  \pgfkeys{/kata/cmd/kata/.cd,#1}
  \section{#2}
  \Ifstr{#3}{}{
```

show if arg 3 is empty but arg 4 was set

```
    \IfNoValueF{#4}{\kata@beltMargin{}{#4}}%
  }{%
```

use arg 3 as fallback if arg 4 was not set

```
    \kata@beltMargin{#3}{\IfNoValueTF{#4}{#3}{#4}}%
  }%
  \setlength{\lineskip}{6pt}\noindent%
  \setcounter{kata@step}{0}%
  \kata@inkatatrue%
  \ignorespaces%
```

there is no harm in a single word/pictogram in the last line to turn off the corresponding penalty

```
  \widowpenalty=0%
```

allow large larger spaces between the pictograms if this avoids drawing a picture outside of the textarea

```
  \sloppy
}{%
```

add hidden steps to the last line. This ensures a uniform looking grid of pictograms

```
  \pgfmathtruncatemacro{\kata@missing}{mod(\pgfkeysvalueof{/kata/cmd/kata/width} - mod(\c@kata@s
  \setcounter{kata@step}{0}%
  \ifnum\kata@missing>0%
    \foreach \i in {1,...,\kata@missing} {%
      \begin{step}[phantom]\end{step} %
    }%
  \fi%
  \kata@inkatafalse%
```

avoid left-aligning the last line. Instead it should be justified in order to obtain a proper grid

```
  \parfillskip=0pt%
  \par%
}
```

`step` (*env.*) this environment always should wrap `kata@step`. It handles the aspects of positioning the pictogram. Arguments: 1 (optional): whether this is the first step of the kata

```
\NewDocumentEnvironment{step}{O{}+b}{%
  \kata@requirekata{\string\step}%
  \pgfkeys{/kata/cmd/step/.cd, first=false, #1}%
  \ifkata@first\par\nointerlineskip\vfil\else\fi%
  \begin{kata@step}%
```

```
    #2
  \end{kata@step}%
  \refstepcounter{kata@step}%
}{}
```

`kata@step` (*env.*) the environment to draw everything else in

```
\NewDocumentEnvironment{kata@step}{}{%
  \kata@requirekata{\string\step}%
  \begin{tikzpicture}[font=\scriptsize]%
    \begin{scope}[local bounding box=output]%
      \kata@head\kata@nothing%
      \kata@insteptrue%
}{
    \kata@instepfalse%
    \end{scope}%
```

Drawing the border on the background has the advantage the rectangle can be filled and colorize the background as well.

```
    \begin{scope}[on background layer]%
      \node[/kata/border,/kata/customize/border] (t1) {};%
      \node[/kata/stepCnt,/kata/customize/stepCnt] (cnt) {\kataStepCnt{kata@step}};%
    \end{scope}%
  \end{tikzpicture}%
}
```

5.2 Specify the Content

`\stance` draw the arrow/path showing the stance Usage:

1. (optional) side to use (L/R)
2. name/text
3. direction

```
\NewDocumentCommand{\stance}{O{}mmO{}}{%
  \pgfkeys{/kata/cmd/content/.cd, flip=false, #4}%
  \kata@requirestep{\string\stance}%
  \pgfmathsetmacro{\kata@dir}{#3+90}%
  \pgfmathsetmacro{\x}{abs(sin(\kata@dir))}%
  \pgfmathsetmacro{\y}{-abs(cos(\kata@dir))}%
  \IfNoValueTF{#1}{\def\kata@nodeContent{}}{\def\kata@nodeContent{#1 }}%
  \draw[/kata/stance={\x}{\y}]
    (head) -- ++(\kata@dir:2)
    node[\ifkata@flip/kata/stanceNodeFlipped\else/kata/stanceNode\fi]
    {\hangafter=1\settowidth\hangindent{\kata@nodeContent}\kata@nodeContent#2}
  ;
}
```

`\waza` draw the arrow/path showing a technique. Arguments:

1. (optional) side to use (L/R)
2. name/text
3. direction

```
\NewDocumentCommand{\waza}{O{}mmO{}}{%
  \kata@requirestep{\string\waza}%
  \pgfkeys{/kata/cmd/content/.cd, flip=false, #4}%
  \IfNoValueTF{#1}{\def\kata@nodeContent{}}{\def\kata@nodeContent{#1 }}%
  \pgfmathsetmacro{\kata@dir}{#3+90}%
  \draw[/kata/waza]
    (head) -- ++(\kata@dir:2)
    node[\ifkata@flip/kata/wazaNodeFlipped\else/kata/wazaNode\fi]
    {\hangafter=1\settowidth\hangindent{\kata@nodeContent}\kata@nodeContent#2}
  ;
}
```

`\dir` draw a short arrow/path indicating the next turn/direction. Arguments:

1. (optional) flip
2. source (Center,angle)
3. dst (angle)

```
\NewDocumentCommand{\dir}{O{}mm}{
  \kata@requirestep{\string\dir}
  \pgfkeys{/kata/cmd/content/.cd, flip=false, #1,}
  \pgfmathsetmacro{\kata@out}{#3+90}
  \Ifstr{#2}{center}{
    \draw[/kata/dir,/kata/customize/dir] (head.center) -- ++(\kata@out:.33em);
  }{
    \pgfmathsetmacro{\kata@in}{#2+90}
    \pgfmathsetmacro{\kata@outDeltaI}{\kata@out-\kata@in}
    \ifkata@flip
      \if -\kata@outDeltaI
        \pgfmathsetmacro{\kata@outDeltaII}{360+\kata@outDeltaI}
      \else
        \pgfmathsetmacro{\kata@outDeltaII}{\kata@outDeltaI-360}
      \fi
    \else
      \pgfmathsetmacro{\kata@outDeltaII}{\kata@outDeltaI}
    \fi
    \draw[/kata/dir,/kata/customize/dir]
      (head.center) ++ (\kata@in:.33em)
      arc[start angle=\kata@in,delta angle=\kata@outDeltaII,radius=.33em]
    ;
  }
}
```

5.3 For user customization

5.3.1 Colors

define xcolors used for coloring elements in the pictograms

```
\RequirePackage{xcolor}
\colorlet{KataColorFeet}{blue}
\colorlet{KataColorWaza}{green}
\colorlet{KataColorDir}{red}
```

5.3.2 Styles

Define some styles for the user to customize the output

Level 1 – Beginner

`\katastyle` A command for setting `/kata/styles` keys similar like `\tikzset`

```
\NewDocumentCommand{\katastyle}{m}{%
  \pgfkeys{/kata/styles/.cd,#1}%
  \ignorespaces%
}
```

```
\pgfkeys{
```

Styles for the pictogram-border (→ `/kata/customize/border`)

```
  /kata/styles/border/.cd,
```

draw arrows to the right on the border, takes the color as argument

```
  right/.style={
    /kata/customize/border/.style={fill=#1!5,draw=#1,/kata/border/arrowRL=-90},
  },
  right/.meaning to context,
```

draw arrows to the left on the border, takes the color as argument

```
  left/.style={
    /kata/customize/border/.style={fill=#1!5,draw=#1,/kata/border/arrowRL=+90},
  },
  left/.meaning to context,
```

draw arrows to the top on the border, takes the color as argument

```
  up/.style={
    /kata/customize/border/.style={fill=#1!5,draw=#1,/kata/border/arrowUD=0},
  },
  up/.meaning to context,
```

draw arrows to the bottom on the border, takes the color as argument

```
  down/.style={
    /kata/customize/border/.style={fill=#1!5,draw=#1,/kata/border/arrowUD=180},
  },
  down/.meaning to context,
```

resets the border modifications

```
reset/.style={/kata/customize/border/.style={}},
reset/.meaning to context,
```

styles to add/adapt the content of a step

```
/kata/styles/step/.cd,
```

draw an indicator for *Kiai*

```
kiai/.style={
  /kata/customize/head/.style={
    append after command={
      \pgfextra{
        \def\base{#1+90} \def\off{30} \def\ext{40} \def\rOff{4pt} \def\len{0.75}
        \draw[shorten <=10pt,CtpRed]
        ($(head)+(\base-\off:\rOff)$) edge[decorate,decoration={bent,aspect=0.3,amplitude=-2.5
        ($(head)+(\base+\off:\rOff)$) edge[decorate,decoration={bent,aspect=0.3,amplitude=+2.5
        \path (tmpA) -- (tmpB) node[midway,sloped,font={\sffamily\scriptsize},CtpRed,draw,deco
      }
    }
  }
},
}
```

Level 2 – Advanced define pgfkeys designed for cosmetic adjustments of the pictograms

`/kata` namespace

```
\pgfkeys{
  /kata/customize/.cd,
```

executed after `/kata/stepCnt`

```
stepCnt/.style={},
stepCnt/.meaning to context,
```

executed after `/kata/head`

```
head/.style={},
head/.meaning to context,
```

executed after `/kata/dir`

```
dir/.style={},
dir/.meaning to context,
```

executed after `/kata/border`

```
border/.style={},
border/.meaning to context,
}
```

Helpers for the border auxilary keys to be used in the `customize/border` style

```
\pgfkeys{
  /kata/border/.cd,
  arrowRL/.style={
    postaction={decorate},decoration={
```

```
      markings,
      mark=at position 0.125
        with {\pgftransformrotate{#1 - 270}\arrow[line width=2pt]{stealth}},
      mark=at position 0.625
        with {\pgftransformrotate{#1 - 270}\arrowreversed[line width=2pt]{stealth}},
    },
  },
  arrowRL/.meaning to context,
  arrowUD/.style={
    postaction={decorate},decoration={
      markings,
      mark=at position 0.375
        with {\pgftransformrotate{#1}\arrow[line width=2pt]{stealth}},
      mark=at position 0.875
        with {\pgftransformrotate{#1}\arrowreversed[line width=2pt]{stealth}},
    },
  },
  arrowUD/.meaning to context,
}
```

5.3.3 Misc

`\kataStepCnt` Formats the counter for the step indicator

```
\NewDocumentCommand{\kataStepCnt}{m}{\arabic{#1}}
```

5.4 Memoization

```
\mmzset{auto={kata@step}{
  memoize,
  csname meaning to context={environment kata@step\space code},
  csname meaning to context={environment kata@step end aux\space},
  csname meaning to context={nothing code},
  csname meaning to context={head code},
  csname meaning to context={stance code},
  csname meaning to context={waza code},
  csname meaning to context={dir code},
```

xcolor stored in another command -> not working as of now

```
  meaning to context={KataColorFeet},
  meaning to context={KataColorWaza},
  meaning to context={KataColorDir},
}}
```

Change History

0.1.0

General: Converted to DTX file 1

Index

Numbers written in italic refer to the page where the corresponding entry is described; numbers underlined refer to the definition; numbers in roman refer to the pages where the entry is used.

A

arrowRL key 6
arrowUD key 6

B

border key 6
border/down key 5
border/left key 5
border/reset key 5
border/right key 5
border/up key 5

C

Colors
  KataColorDir 5
  KataColorFeet 5
  KataColorWaza 5

D

dir key 6

F

first key 3
flip key 3

H

head key 6

K

KataColorDir color 5
KataColorFeet color 5
KataColorWaza color 5

Keys
  /kata/border/
    arrowRL 6
    arrowUD 6
  /kata/cmd/content/
    flip 3
  /kata/cmd/kata/
    width 3
  /kata/cmd/step/
    first 3
    phantom 3
  /kata/customize/
    border 6
    dir 6
    head 6
    stepCnt 6
  /kata/styles/
    border/down 5
    border/left 5
    border/reset 5
    border/right 5
    border/up 5
    step/kiai 5

P

phantom key 3

S

step/kiai key 5
stepCnt key 6

W

width key 3